Fine-Tuning TinyLlama on Databricks AI Runtime — Deployment Runbook

Multi-node, multi-GPU distributed training with Ray Train + PyTorch Lightning FSDP, packaged as a Databricks Asset Bundle (DAB).

1. Overview

This Databricks Asset Bundle (DAB) deploys a single Databricks Job that fine-tunes the TinyLlama model across multiple GPU nodes on Databricks AI Runtime (serverless GPU). It uses Ray Train to launch one training worker per GPU across all nodes, and PyTorch Lightning with FSDP (Fully Sharded Data Parallel) for memory-efficient sharded training.

The entire job — compute, environment, code, and schedule — is defined declaratively and deployed with the Databricks CLI, so it is reproducible across environments.

How a run executes

1. AI Runtime provisions the GPU nodes (for example, 24 GPUs = 3 × `GPU_8xH100`) and builds the Python environment.
2. It runs `command.sh` on every node. Node 0 starts the Ray head and the training driver; the other nodes join as Ray workers.
3. The driver waits for all GPUs to join, then starts Ray Train, which runs the training function once per GPU with Lightning FSDP.
4. Checkpoints are written to a Unity Catalog Volume. When the driver finishes, the cluster tears down cleanly.

2. Prerequisites

- Databricks CLI v1.20.0 or later. Check with: `databricks --version`
- A Databricks workspace with AI Runtime / serverless GPU enabled.
- A Unity Catalog Volume for checkpoints.
- Recommended for production: a service principal to run the job.
- An authenticated CLI profile: `databricks auth login --host <workspace-url> --profile <PROFILE>`

> **IMPORTANT**
> Earlier CLI versions silently ignore the `ai_runtime_task` fields. Use v1.20.0 or later.

Checkpoint Volume (Unity Catalog)

`ray_storage_path` in `training_config.json` points at a Unity Catalog Volume path, for example `/Volumes/<catalog>/<schema>/<volume>/ray_results`.

> **NOTE**
> Only the `catalog`, `schema`, and `volume` must already exist in Unity Catalog — they are governed objects and the job cannot create them. Pointing at a non-existent catalog/schema/volume fails at task init with an opaque `INTERNAL_ERROR` (and no logs). Everything below the volume — `ray_results/`, the run-name folder, and the checkpoint directories — is created automatically by Ray Train on the first run, so you do not need to pre-create it.

The `run_as` service principal needs these Unity Catalog privileges on the checkpoint Volume:

Object	Privilege	Purpose
Catalog	`USE CATALOG`	Traverse into the catalog
Schema	`USE SCHEMA`	Traverse into the schema
Volume	`USE VOLUME`	Access the volume
Volume	`WRITE VOLUME`	Create directories and write checkpoints

Granting `USE CATALOG` / `USE SCHEMA` / `USE VOLUME` / `WRITE VOLUME` at the catalog level cascades to all schemas and volumes within it. `READ VOLUME` alone is not sufficient — the job writes checkpoints.

3. Repository contents

The bundle layout at a glance:

```
air_ray_fsdp_llama_dabs/
│
├── databricks.yml            Bundle definition: job, AI Runtime task, compute,
│                             environments (targets), and permissions — the main file
├── variables.yaml            Deployment & infrastructure variables; per-environment overrides
├── command.sh                Per-node entrypoint: starts Ray (head on node 0, workers
│                             elsewhere) and launches the training driver
│
├── src/                      Shipped to every node (packaged into dist/code.tgz)
    ├── train_llama_ray_fsdp.py   Training driver (Ray Train + Lightning FSDP)
    ├── training_config.json      All training settings + resume_checkpoint (edit to retune)
    └── training_examples.json    Instruction-tuning dataset (24 examples)
```

File	What it is
`databricks.yml`	The bundle definition and main file. Declares the job, the AI Runtime task, compute, the Python environment, the environments (targets), and permissions.
`variables.yaml`	Deployment and infrastructure variables: names, service principal, per-environment root paths, package versions, and Ray bootstrap settings. Edit these per environment.
`command.sh`	The per-node entrypoint. Starts Ray (head on node 0, workers elsewhere) and launches the training driver.
`src/train_llama_ray_fsdp.py`	The training driver (Ray Train + Lightning FSDP). Reads its settings from the two JSON files below.
`src/training_config.json`	All training settings: model, hyperparameters, FSDP configuration, Ray settings, and `resume_checkpoint`. Edit this to retune.
`src/training_examples.json`	The instruction-tuning dataset (24 examples). Replace with your own data.

4. How configuration works (two channels)

AI Runtime limits job environment variables to 20 entries and 512 characters each, so configuration is split into two channels:

- **Small scalars → environment variables** (declared in `variables.yaml`, mapped in `databricks.yml`). Only the few values `command.sh` needs: `TRAINING_SCRIPT`, `RAY_HEAD_PORT`, `RAY_JOIN_RETRIES`, `RAY_JOIN_SLEEP_SECONDS`.
- **Training configuration and dataset → JSON files** shipped inside the code artifact: `src/training_config.json` and `src/training_examples.json`. The driver reads these from disk.

To retune training or change the data, edit the JSON files in `src/` — no change to `databricks.yml` or Python is needed.

> **NOTE**
> `NODE_RANK`, `MASTER_ADDR`, `MASTER_PORT`, `NUM_NODES`, `LOCAL_WORLD_SIZE`, `WORLD_SIZE`, and `CODE_SOURCE_PATH` are provided automatically by AI Runtime at runtime. You do not set them.

Where `command.sh` gets its variables at run time (and the service-principal caveat)

The deployed job has two separate, similarly named fields. Do not confuse them:

Field in the job definition	What it holds	Shown in the Jobs UI?
`environments`	The compute environment: `base_environment` plus the pip dependencies	Yes — this is the "Configure environment" panel
`environment_variables`	The name/value pairs `command.sh` reads (`TRAINING_SCRIPT`, `RAY_HEAD_PORT`, `RAY_JOIN_RETRIES`, `RAY_JOIN_SLEEP_SECONDS`)	No — the AI Runtime (Preview) UI does not render this block

Both are stored in the deployed job. The `environment_variables` block is simply not surfaced in that UI panel — view it with `databricks jobs get <job-id>` or in the run detail.

> **IMPORTANT**
> AI Runtime injects the job's `environment_variables` into the process only when `run_as` is a user. Under a service principal it applies none of them (the launcher logs "Applying 0 user env var(s)"). So on a service-principal job, `command.sh` does not receive `RAY_HEAD_PORT`, `TRAINING_SCRIPT`, and the others from the job definition — it falls back to its own built-in defaults, shown below.

command.sh self-defaults every value it needs:

```
RAY_HEAD_PORT="${RAY_HEAD_PORT:-6379}"
RAY_JOIN_RETRIES="${RAY_JOIN_RETRIES:-30}"
RAY_JOIN_SLEEP_SECONDS="${RAY_JOIN_SLEEP_SECONDS:-5}"
TRAINING_SCRIPT="${TRAINING_SCRIPT:-train_llama_ray_fsdp.py}"
```

At run time, the variables the code uses come from three places:

1. **Injected by AI Runtime, always present** (even for a service principal): `NODE_RANK`, `MASTER_ADDR`, `MASTER_PORT`, `NUM_NODES`, `LOCAL_WORLD_SIZE`, `WORLD_SIZE`, `CODE_SOURCE_PATH`.
2. **command.sh's own defaults** — what actually supplies `RAY_HEAD_PORT`, `TRAINING_SCRIPT`, and the rest on a service-principal job.
3. **The job's environment_variables block** — stored in the definition, but injected only for a user `run_as` (for example, the `air` CLI); inert for a service-principal run.

> **NOTE**
> Keep the defaults in `command.sh` and the `environment_variables` block (in `variables.yaml` / `databricks.yml`) in sync: the first serves service-principal runs, the second serves user runs.

5. Set up for your workspace

Configure workspace values

Before the first deploy, set these values:

Where	Setting	Example
`databricks.yml` (targets)	`workspace.host` — your workspace URL (must be a literal; variable interpolation is not allowed here)	`https://<your-workspace>.cloud.databricks.com`
`variables.yaml` (per target)	`workspace_root_path` — the workspace folder the bundle deploys into	`/Workspace/prod`
`variables.yaml` (per target)	`command_path` — must equal `<root_path>/files/command.sh`	`/Workspace/prod/files/command.sh`
`variables.yaml` (per target)	`service_principal_application_id` — the service principal the job runs as	`<service-principal-application-id>`
`src/training_config.json`	`ray_storage_path` — UC Volume path for checkpoints	`/Volumes/<catalog>/<schema>/<volume>/ray_results`

Grant permissions

- Grant the service principal `WRITE_VOLUME` on the checkpoint Volume, so it can write checkpoints.
- Grant the deploying identity the `servicePrincipal.user` role on the service principal, so `run_as` can bind at deploy time.
- The bundle's `permissions` block grants the service principal (and the deployer) `CAN_MANAGE` on the deployed workspace folder, so the job — running as the service principal — can read the uploaded code. Keep this block. Without it, a non-shared `root_path` causes the error "User lacks read access to file: .../command.sh".

6. Deploy

Validate and deploy

Validate, then deploy:

```
databricks bundle validate --target prod --profile <PROFILE>
databricks bundle deploy  --target prod --profile <PROFILE>
```

Deploy builds the code archive, uploads it and the bundle files to the workspace, and creates or updates the job.

Enable the schedule

The bundle ships with the schedule paused (`schedule_pause_status: PAUSED`). To enable the cron, set it to `UNPAUSED` in `variables.yaml` and redeploy.

7. Run the job

```
databricks bundle run llama_finetuning --target prod --profile <PROFILE>
```

Use the resource key `llama_finetuning`, not the display name. You can also click Run Now on the job page in the workspace UI.

Scale the GPU count

Scale GPUs without editing files by passing `accelerator_count` (a multiple of 8; 8 GPUs per node):

```
databricks bundle run llama_finetuning --target prod --profile <PROFILE>
--var="accelerator_count=16"
```

8. Read the logs

> **IMPORTANT**
> A failed `ai_runtime_task` reports only a generic `INTERNAL_ERROR` in the Jobs UI and API — the real traceback is not shown there. The per-node stdout/stderr is captured as MLflow run artifacts.

Read the per-node logs

To read them:

1. Open the MLflow experiment (default location: `/Users/<deployer>/air-ray-fsdp-llama`) and find the run for your job run.
2. List and read the per-node log artifact:

```
databricks api get
"/api/2.0/mlflow/artifacts/list?run_id=<RUN_ID>&path=logs/node_0" --profile
<PROFILE>

databricks api get
"/api/2.0/mlflow/get-artifact?run_uuid=<RUN_ID>&path=logs/node_0/logs-0.chunk
.txt" --profile <PROFILE>
```

`node_0` is the driver and is the most useful. Other nodes are `node_1`, `node_2`, and so on.

9. Checkpointing and resume

Where checkpoints are written

Checkpoints are written to the UC Volume at `ray_storage_path`, under `ray_results/<ray_train_run_name>/`. Retention is bounded by `checkpoints_to_keep`.

Automatic resume

If that run directory already contains a checkpoint, a rerun resumes from the latest one instead of retraining. For a fresh run, use a new `ray_train_run_name`.

Explicit resume

Set `resume_checkpoint` to a checkpoint directory (the folder that contains `checkpoint.ckpt`) and use a new `ray_train_run_name`. Resume restores the epoch counter and continues only the remaining epochs up to `max_epochs`.

10. Environments (targets)

The bundle defines three environments — `prod` (default), `stage`, and `test` — each isolated by its own `workspace.root_path` and its own `run_as` service principal. Deploy or run any of them by changing `--target`:

```
databricks bundle deploy --target stage --profile <PROFILE>
```

> **NOTE**
> `stage` and `test` ship with placeholder service principal IDs. Replace them with real IDs before deploying those environments.

11. Important notes

- **CLI version:** use v1.20.0 or later, or the `ai_runtime_task` configuration is silently dropped.
- **Keep the AI Runtime task minimal:** do not add an explicit MLflow experiment directory or artifact location on a Volume path. It causes the task to fail at initialization with no logs.
- **Service principal access:** the SP needs `WRITE_VOLUME` on the checkpoint Volume and read access (via the `permissions` block) to the deployed code folder. Under a service-principal `run_as`, the job's environment variables are not applied, so `command.sh` self-defaults every value it needs.
- **Gradient clipping** is handled through FSDP inside the model. Do not rely on Lightning's default clipping; it is unsupported with FSDP precision.
- **Do not use Ray's removed resume_from_checkpoint API.** Resume is handled through Lightning's `ckpt_path`.

12. Troubleshooting

Symptom	Likely cause	Fix
`INTERNAL_ERROR` with no traceback	Normal for AI Runtime failures	Read the real logs from MLflow artifacts (Section 8).
"User lacks read access to .../command.sh"	The `run_as` service principal cannot read the deployed folder	Ensure the `permissions` block grants the SP `CAN_MANAGE`, then redeploy.
Task fails at initialization, no logs	An explicit MLflow artifact location was set on a Volume path	Remove it; keep the AI Runtime task minimal.
"unbound variable" in `command.sh`	Environment variables are not applied under a service-principal `run_as`	`command.sh` must self-default values via `${VAR:-default}`.
Run stuck in "Blocked"	Concurrency: `max_concurrent_runs` is 1 and a run is already active	Wait for or cancel the active run.

Cannot write checkpoints	SP is missing `WRITE_VOLUME` on the Volume	Grant `WRITE_VOLUME` on the checkpoint Volume.
Multi-node run hangs for a long time after training already finished	Worker nodes used `ray start --block`, which never returns when node 0 stops the Ray head	Workers poll `ray status` and exit when the head disappears — see "Deep dive" below

Deep dive: a multi-node job that hangs after training completes (the --block trap)

Symptom. On multi-node runs only, training completes successfully — node 0 finishes and exits with code 0 — but the job stays in Running for a very long time and is eventually killed at the task timeout (for example, 2 hours), even though the actual work was already done. Single-node runs are never affected.

Why it happens. In a multi-node run, node 0 hosts the Ray head and runs the training driver; the other nodes are Ray workers. The original worker command started Ray with the `--block` flag, which keeps the worker process in the foreground indefinitely. That seems correct — you want the worker to stay up during training — but it has a fatal flaw: when training ends, node 0 stops the Ray head and exits, and the workers' blocking `ray start --block` call does not return when the head goes away. Each worker's `command.sh` stays frozen on that line. Because AI Runtime only finishes the task once every node has finished, the entire job hangs in teardown until the timeout, even though training itself succeeded.

A simple analogy. Think of a conference call. Node 0 is the host; the workers are participants. When the meeting is over, the host hangs up — but the participants' phones were set to "stay on the line forever" and never notice the host left. The call never officially ends, so everyone is stuck until the system force-disconnects them hours later.

How it was solved. The workers no longer use `--block`. Each worker now starts Ray in the background, joins the head, and then watches the cluster by polling `ray status` in a loop. While the head is alive the check passes; when node 0 finishes and stops the head, the check starts failing, and after three consecutive failures — a guard so a brief network blip during training does not drop a worker by mistake — the worker runs `ray stop` and exits cleanly. Now, the moment training finishes, every node notices and exits, so the job terminates immediately instead of hanging until the timeout.

Before (caused the hang)

```
ray start --address="${MASTER_ADDR}:${RAY_HEAD_PORT}"
--num-gpus="$GPUS_PER_NODE" --block
```

After (clean teardown)

```
ray start --address="${MASTER_ADDR}:${RAY_HEAD_PORT}"
--num-gpus="$GPUS_PER_NODE"

misses=0
while true; do
  if ray status >/dev/null 2>&1; then
    misses=0                      # head still alive
  else
    misses=$((misses + 1))
    [ "$misses" -ge 3 ] && break   # head gone 3 times in a row -> stop
waiting
  fi
  sleep "$RAY_JOIN_SLEEP_SECONDS"
done
ray stop >/dev/null 2>&1 || true
```

13. Clean up

```
databricks bundle destroy --target prod --profile <PROFILE>
```

This removes the job and the deployed bundle files for that environment. Checkpoints already written to the UC Volume are not deleted.